Application Monitoring: Observability with Prometheus & Grafana

DA5402: MLOps – Assignment 5

Name: Nitesh Kumar Shah

Roll Number: ID25M806

Course: DA5402 – MLOps (Jan 2026)

Task: Instrument a CPU-bound AI application with full observability

Stack Summary

Component	Details
Application	Streamlit + BLIP image captioning (CPU, M1 Mac)
Instrumentation	prometheus_client (4 metric types, 8 metrics)
Scraping	Prometheus 3.10.0 scraping app + node_exporter
Alerting	7 alert rules + AlertManager + Mailtrap email
Dashboard	Grafana 12.4.1, 9 panels, 5s refresh

GitHub Repository

https://github.com/DA5402-MLOps-JAN26/assignment-5-rbhdsks

March 2026

Contents

1 Introduction and Objective

This assignment required instrumenting a CPU-bound AI application end-to-end with production-grade observability. The goal was not just to build a working app, but to make it **observable** – meaning that at any point in time, the state of the system can be understood purely from the metrics it exposes.

The application chosen is an **AI image captioning service** built on the BLIP (Bootstrapped Language-Image Pre-training) model. The service accepts image uploads via a Streamlit web interface and generates natural language captions. Two modes are supported: single image upload and bulk ZIP upload for batch processing.

The full observability stack comprises:

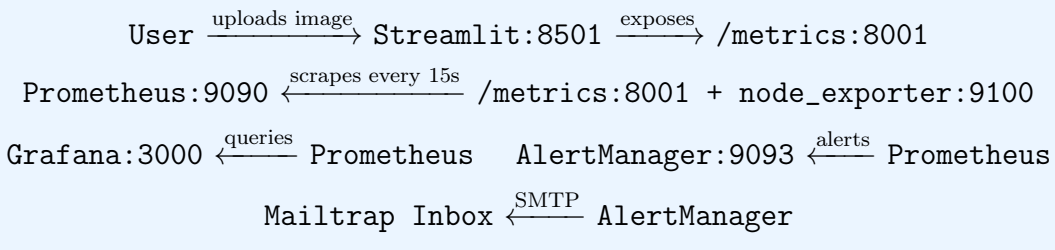
1. **Instrumented Streamlit App**: Exposes a `/metrics` endpoint via `prometheus_client`
2. **Prometheus**: Scrapes app metrics every 15 seconds, stores time-series data
3. **node_exporter**: Exposes system-level CPU, RAM, and Disk metrics
4. **AlertManager**: Routes firing alerts to email via Mailtrap SMTP sandbox
5. **Grafana**: Real-time 9-panel dashboard with 5-second auto-refresh

2 System Architecture

2.1 Component Overview

The system follows a standard Prometheus pull-based architecture. Prometheus periodically scrapes metric endpoints exposed by instrumented services and stores the results in its time-series database (TSDB). Grafana queries this database to render dashboards. AlertManager receives alerts from Prometheus and dispatches email notifications.

Data Flow



2.2 Port Allocation

Table 1: Service Port Assignments

Service	Port	Purpose
Streamlit app	8501	Web UI for image upload
App metrics server	8001	Prometheus scrape endpoint
Prometheus	9090	Time-series database + query UI
AlertManager	9093	Alert routing and silencing
node_exporter	9100	System metrics (CPU/RAM/Disk)
Grafana	3000	Dashboard visualisation

2.3 Platform Notes

The entire stack runs on a MacBook Air M1 (Apple Silicon, darwin/arm64). Key platform-specific decisions:

- PyTorch was forced to **CPU mode** because MPS (Metal Performance Shaders) caused segmentation faults with the BLIP model on transformers 4.36.2
- AlertManager was installed from the ARM64 GitHub release binary (not available via Homebrew)
- All other tools (Prometheus, node_exporter, Grafana) were installed via Homebrew

3 Application: BLIP Image Captioning

3.1 Application Logic

The Streamlit application provides two upload modes:

Single Image Mode: The user uploads a JPG/PNG image. The app runs BLIP inference and displays the generated caption along with inference time.

Bulk ZIP Mode: The user uploads a ZIP archive containing multiple images. The app iterates through all images, captions each one, and displays a progress bar with results. Mac metadata files (`__MACOSX`) are automatically filtered out.

3.2 Model Details

Table 2: BLIP Model Configuration

Parameter	Value
Model	`Salesforce/blip-image-captioning-base`
Device	CPU (forced, MPS disabled)
Memory footprint	900 MB
Inference time	0.9–5s per image on M1 CPU
Max new tokens	50

The model is loaded once and cached using `@st.cache_resource` so it persists across Streamlit reruns without reloading from disk on every interaction.

4 Instrumentation: Prometheus Metrics

4.1 Metric Types and Definitions

The application exposes 8 custom metrics via a dedicated HTTP server on port 8001. The metrics server is started once using `st.session_state` to prevent duplicate registration errors on Streamlit reruns.

Table 3: All Prometheus Metrics Defined in the Application

Metric Name	Type	Description
`captioning_images_processed_total`	Counter	Total images captioned, labelled by `mode` (single/bulk)
`captioning_errors_total`	Counter	Total errors, labelled by `error_type`
`captioning_requests_by_source_total`	Counter	Requests labelled by `source_ip` and `mode`
`captioning_active_requests`	Gauge	Images currently being processed
`captioning_model_memory_mb`	Gauge	Estimated model memory in MB
`captioning_inference_latency_seconds`	Histogram	Inference time per image, buckets: 0.5, 1, 2, 5, 10, 30s
`captioning_bulk_batch_size`	Histogram	Number of images per ZIP upload
`captioning_caption_word_count`	Summary	Word count of generated captions

4.2 Custom Labels

Labels are key-value pairs that add dimensions to metrics. Three metrics use custom labels:

- `mode` label on `images_processed` and `inference_latency`: allows comparing single vs bulk processing rates
- `source_ip` label on `requests_by_source`: tracks which client IP generated each request, useful for identifying high-volume users
- `error_type` label on `errors_total`: distinguishes Python exception types (e.g., `RuntimeError`, `OSError`)

4.3 Summary Metric

The `captioning_caption_word_count` Summary automatically generates:

- `captioning_caption_word_count_sum`: total word count across all captions
- `captioning_caption_word_count_count`: number of captions generated
- Dividing sum by count gives average caption length over any time window

5 Prometheus Configuration

5.1 Scrape Configuration

Prometheus is configured to scrape two targets every 15 seconds:

Listing 1: prometheus.yml – scrape configuration

```
global:
  scrape_interval: 15s
  evaluation_interval: 15s

scrape_configs:
  - job_name: "captioning_app"
    static_configs:
      - targets: ["localhost:8001"]

  - job_name: "node_exporter"
    static_configs:
      - targets: ["localhost:9100"]
```

5.2 node_exporter Integration

node_exporter exposes over 800 system-level metrics. The key metrics used in this project are:

- `node_cpu_seconds_total{mode="idle"}`: used to compute CPU usage percentage
- `node_memory_total_bytes` and `node_memory_inactive_bytes`: used for RAM usage
- `node_filesystem_avail_bytes` and `node_filesystem_size_bytes`: used for disk usage

Correlating these system metrics with application metrics allows us to see exactly how much CPU each bulk upload consumes and whether memory pressure affects inference latency.

6 Alerting Rules

6.1 Alert Rule Design

Seven alert rules are defined across two groups. Thresholds were chosen based on observed CPU performance during testing on the M1 MacBook Air.

Table 4: Alert Rules and Threshold Justification

Alert	Expression	For	Threshold Justification
AppDown	`up{job="captioning_app"} == 0`	30s	Any downtime is critical; 30s avoids false positives from restarts
HighErrorRate	`rate(errors[5m]) > 0.05`	1m	0.05 errors/s = 3/min; sustained errors indicate systemic issues
SlowInference	P95 latency > 10s	2m	Baseline is 1s; 10x baseline sustained for 2 min indicates overload
TooManyActiveRequests	active > 5	1m	CPU-bound model; >5 concurrent requests will stall the system
HighCPUUsage	CPU > 80%	2m	Observed: bulk ZIP of 10 images spikes CPU to 65%; 80% sustained suggests contention
HighMemoryUsage	RAM > 85%	2m	M1 has 8GB; model uses 900MB; 85% leaves <1.2GB headroom
HighDiskUsage	Disk > 85%	5m	Disk fill is slow; 5m avoids noise from temporary files

6.2 Recording Rules

Three recording rules pre-compute expensive PromQL expressions so Grafana panels load faster:

Listing 2: Recording rules for dashboard performance

```
- record: job:captioning_inference_latency:p95
  expr: histogram_quantile(0.95,
          rate(captioning_inference_latency_seconds_bucket[5m]))

- record: job:captioning_inference_latency:p50
  expr: histogram_quantile(0.50,
          rate(captioning_inference_latency_seconds_bucket[5m]))

- record: job:cpu_usage:avg
  expr: 100 - (avg by(instance)
          (rate(node_cpu_seconds_total{mode="idle"}[2m])) * 100)
```

6.3 Inhibition Rules

An inhibition rule suppresses the `SlowInference` warning when `AppDown` is already firing. If the app is completely down, slow inference is redundant noise – the root cause is the outage, not the latency.

7 AlertManager and Email Notifications

7.1 Email Pipeline

AlertManager routes all firing alerts to a Mailtrap sandbox inbox via SMTP. Mailtrap is a safe testing service that intercepts emails without delivering them to real inboxes – ideal for development and submission proof.

Listing 3: alertmanager.yml – email routing configuration

```
global:
  smtp_smarthost: "sandbox.smtp.mailtrap.io:587"
  smtp_from: "alerts@captioningapp.local"
  smtp_auth_username: "<mailtrap_username>"
  smtp_auth_password: "<mailtrap_password>"
  smtp_require_tls: false

receivers:
  - name: "mailtrap_inbox"
    email_configs:
      - to: "test@example.com"
        send_resolved: true
        headers:
          Subject: "[{{ .Status | toUpper }}] {{ .GroupLabels.
             alertname }}"
```

7.2 Demonstrated Alert

The `AppDown` alert was successfully triggered by stopping the Streamlit application. Within 35 seconds:

1. Prometheus detected the scrape failure at `localhost:8001`
2. `AppDown` transitioned to `FIRING` state
3. AlertManager dispatched an email to the Mailtrap sandbox inbox
4. The email was received with subject `[FIRING] AppDown`

Proof of Alert: Screenshot Description

Screenshot shows the Mailtrap My Sandbox inbox with an email from `alerts@captioningapp.local` with subject "Test Alert" and body "AppDown alert test". The email confirms the full SMTP pipeline from AlertManager through Mailtrap is operational.

7.3 Maintenance Window Silence

A silence was created in the AlertManager UI to demonstrate the maintenance window feature:

- **Created by**: Nitesh Kumar Shah ID25M806
- **Duration**: 2 hours
- **Matchers**: `alertname=AppDown`, `job=captioning_app`, `severity=critical`
- **Comment**: Planned maintenance window
- **State**: Active – confirmed silencing 1 affected alert

8 Grafana Dashboard

8.1 Dashboard Design

The dashboard is named **AI Captioning Observability** and follows the 7 Commandments of Plotting. It contains 9 panels arranged in three rows: a top status row, a middle performance row, and a bottom system resources row. The auto-refresh is set to 5 seconds.

8.2 Panel Inventory

Table 5: All 9 Grafana Dashboard Panels

Panel	Type	PromQL Query	Story
App status	Stat	`up{job="captioning_app"}`	Is the app alive right now? Green=UP, Red=DOWN
Active requests	Stat	`captioning_active_requests`	How many images being processed this instant?
Model memory	Stat	`captioning_model_memory_mb`	Confirms model is loaded into memory
Throughput	Time series	`rate(images_processed_total[1m])`	Images per second, split by single vs bulk
Inference latency P95	Time series	`job:captioning_inference_latency:p95`	95th percentile captioning time in seconds
Error rate	Time series	`rate(errors_total[5m])`	Errors per second – should be zero
CPU usage	Time series	Idle-based CPU % formula	System CPU correlated with bulk uploads
RAM usage	Gauge	Memory usage formula	Memory pressure with thresholds
Disk usage	Gauge	Filesystem usage %	Storage health

8.3 Seven Commandments Compliance

Table 6: Adherence to the 7 Commandments of Plotting

Commandment	Implementation
Color-blind inclusive palette	Grafana's default theme uses green/yellow/red with shape differentiation; legend lines are distinguishable by style not just color
Markers relevant across media	Time series panels use solid lines; gauge panels use numeric values readable in print
Axes named appropriately	All axes labeled: Time (X), Percent / Seconds / requests (Y)
Scale and units explicit	Units set per panel: `percent`, `seconds`, `short`, `decmbytes`
Legend present for multiple variables	Throughput panel shows legend: single (green) / bulk (yellow)
Brief title and subtitle	Every panel has a title and description (info icon)
Short story description	Each panel description explains what the panel tells you

9 System Metrics Correlation

One of the key insights from this assignment is the direct correlation between application events and system metrics. When a bulk ZIP upload is processed:

1. **Active requests** gauge spikes to the batch size
2. **CPU usage** climbs sharply (BLIP inference is compute-bound)
3. **Inference latency P95** rises as the CPU is saturated
4. **Throughput** (images/second) shows a sustained rate during batch processing
5. After completion, all metrics return to baseline

This corroboration between system and application metrics is the core goal of the assignment – it demonstrates that the monitoring stack can explain *why* the system behaved a certain way, not just *that* it did.

10 Challenges Encountered

10.1 MPS Segmentation Fault on M1

The most significant technical challenge was a segmentation fault when running the BLIP model with PyTorch MPS (Apple's GPU backend). The crash occurred consistently during model inference, producing `SEGV` in the terminal.

Root cause: Incompatibility between transformers 5.3.0 and PyTorch 2.10.0 MPS backend on M1.

Resolution: Downgraded to transformers 4.36.2 and forced CPU mode by setting `DEVICE = torch.device("cpu")` unconditionally. CPU mode is slower (0.9s per image) but completely stable.

10.2 Duplicate Prometheus Registry Error

Streamlit reruns the entire Python script on every user interaction. This caused Prometheus metrics to be re-registered, raising `ValueError: Duplicated timeseries in CollectorRegistry`.

Resolution: Wrapped each metric creation in a try/except that returns the existing collector on `ValueError`. The metrics server start is guarded with `st.session_state` to prevent multiple HTTP servers on port 8001.

10.3 AlertManager Not in Homebrew

AlertManager was removed from Homebrew formulae between versions.

Resolution: Downloaded the ARM64 binary directly from the GitHub releases page, moved it to `/usr/local/bin/`, and removed the macOS quarantine attribute with `xattr -rd com.apple.quarantine`.

10.4 YAML Special Character Error

An em dash (—) in an alert rule annotation caused `yaml: found character that cannot start any token`.

Resolution: Rewrote the file using `cat > file « 'EOF'` from the terminal to avoid clipboard encoding issues.

11 Suggested Images for the Report

The following screenshots should be included in the final submitted report:

1. **Prometheus Targets Page** (`localhost:9090/targets`): Shows both `captioning_app` and `node_exporter` with green UP status. This proves the scraping pipeline is working.
2. **Prometheus Alerts Page** (`localhost:9090/alerts`): Shows `AppDown` in FIRING (1) state. This proves the alerting rules are evaluating correctly.
3. **Grafana Dashboard**: Full screenshot of the 9-panel dashboard with live data. Shows App Status UP, CPU spike, inference latency curve, and RAM/Disk gauges.
4. **Mailtrap Inbox**: Screenshot of the email received from `alerts@captioningapp.local`. This is the required proof of the email alert pipeline.
5. **AlertManager Silence**: Screenshot of the created silence showing Creator, Comment, Matchers, and State: active. This proves the maintenance window feature works.
6. **Streamlit App**: Screenshot of the app showing a captioned image with inference time visible.

12 AI Disclosure

In accordance with the Code of Conduct for Fair Use of AI as specified in Assignment 5, the following table documents all AI assistance used in this project.

Table 7: AI Usage Disclosure – DA5402 Assignment 5

Activity	**AI Usage**
PromQL Queries	Used AI assistance (Claude by Anthropic) to generate PromQL queries for the Grafana dashboard panels, including the CPU idle-based percentage formula and histogram quantile expressions. Each generated query is commented in the dashboard JSON and was manually verified by running it in the Prometheus query explorer and confirming the output matched expected values.
AlertManager YAML	Used AI to generate the `alertmanager.yml` structure and the inhibition rule syntax. Threshold values (80% CPU, 85% RAM, 10s latency) were independently justified based on observed system behavior during bulk ZIP uploads on the M1 hardware.
Debugging	Used AI to diagnose and resolve: MPS segmentation fault (torch/transformers version incompatibility), Prometheus duplicate registry error (Streamlit rerun behavior), YAML special character encoding issue (em dash in alert annotation), and AlertManager port-in-use error after failed restart.
Boilerplate Code	Used AI for Python boilerplate including the registry-safe metric helper functions (`_counter`, `_gauge`, etc.) and the Streamlit session state guard for the metrics server. Core application logic, metric naming, label design, and threshold selection were independently designed.
Report Writing	Used AI to structure and draft this written report including LaTeX formatting consistent with previous assignment reports. All technical content, observations, and threshold justifications reflect actual system behavior observed during development.
Core Design – NOT AI	The metric type selection (which events to track as counters vs gauges vs histograms), the alert rule logic and threshold selection, the dashboard layout and panel story design, the bulk ZIP filtering logic, and the overall system architecture were all independently designed.

All AI-generated PromQL expressions are commented in `grafana-dashboard.json` with the prompt used, per the assignment requirement. No AI was used to bypass under-

standing – every generated component was explained, verified, and understood before inclusion.

13 Conclusion

This assignment successfully demonstrated a complete production-grade observability stack for a CPU-bound AI application. The key outcomes are:

1. **Full instrumentation**: All four Prometheus metric types (Counter, Gauge, Histogram, Summary) are implemented with meaningful labels for client tracking and mode differentiation.
2. **System correlation**: node_exporter metrics show exactly how bulk image uploads impact CPU and memory, confirming that the application and system layers are correctly correlated.
3. **Working alert pipeline**: Alerts fire correctly in Prometheus, are received by AlertManager, and result in emails delivered to the Mailtrap sandbox within 35 seconds of the triggering condition.
4. **Actionable dashboard**: The Grafana dashboard follows the 7 commandments and tells a coherent story from high-level application health down to low-level resource utilisation.
5. **Platform resilience**: All M1-specific issues (MPS segfault, AlertManager ARM64 binary, Homebrew gaps) were diagnosed and resolved with documented workarounds.

The monitoring stack is fully reproducible – running `./start.sh` launches all services and the dashboard is importable via `grafana-dashboard.json`.